Assignment 03 Synchronisation

1 Synchronisation

1. **The principle of process isolation in an operating system means that processes must not have access to the address spaces of other processes or the kernel. However, processes also need to communicate.**
   a. Give an example of such communication.
   An example of interprocess communication (IPC) is the well known transport protocols TCP/UDP sockets

b. How does this communication work?
Works by creating a memory segment that can be accessed by multiple processes. A process (parent) creates the shared memory segment using syscall `shmget()` and assign it the identifier `shmid`. The processes that need access to the shared memory attaches themselves to this segment using syscall `shmat()`. Now the processes can read/write to the shared memory. When processes no longer need access to the shared memory, they can detach from the segment using syscall `schmtl()`. Once all processes are detached the segment can be removed from the system using `schmctl`.

c. What problems can result from inter-process communication?
There are three problems when using cooperative threads if not handled correctly:
1 Race condition: Program execution depends on the interleaving of the thread's access to shared data. The threads run a race between each other and the result depends on who wins the race

2 Program execution prone to be nondeterministic. Different executions of the same program gives different results. Affected by factors such as scheduler makes different scheduling decision.

3 Compiler reorder instructions to improve performance, but this reordering is not visible to single-threaded programs, but can become visible when multiple processes interact through shared data.
2. **What is a critical region? Can a process be interrupted while in a critical region? Explain.**
Critical region is a segment of code where a constraint is applied so that only one thread can execute at once (atomically accesses shared memory). The purpose is to avoid problems such as race conditions and data inconsistency. Because it accesses shared memory atomically,

other threads or processes can not access that shared memory, thus implies that the process that executes the critical region is isolated.

A thread running a critical region can be interrupted by the OS because critical region only implies that no thread can execute the same critical region at once. Interrupts can be disabled if wanted.

3. **Explain the difference between busy waiting (polling) versus blocking (wait/signal) in the context of a process trying to get access to a critical section.**

Both are synchronization techniques.

Busy-waiting is when a threads (process) spins in a loop waiting for a concurrent event to occur/check if the condition is satisfied, consuming CPU cycles while waiting to access a critical section for execution.

Wait/signal is when one process pauses execution until it receives a signal from another thread allowing for coordinate actions. Used to prevent race conditions

So when busy-wait always ask for the lock, wait/signal ask only when receiving a signal.

https://www.geeksforgeeks.org/operating-systems/busy-waiting-in-os/ and the book

4. **What is a race condition? Give a real-world example.**

   When the behaviour of a program depends on interleaving of operations of different threads. So the threads run a race between their operations, and the result of the program execution is depended on who wins the race.

Example:
Two people both have access to the same bank account and both try to pay for something costing 1kr. The system check their balance and validates if there is enough money for both payments. If there is not proper synchronization, the system allows both transaction to go through even though the bank account amount is only 1kr.

```c
#include <stdlib.h>
#include <pthread.h>

int balance = 1;

void pay(int cost){
    balance -= cost; // critical region, accessing and modifiing the same objec
}

int main(){
    int threadCount = 2;
    pthread_t thread[threadCount];
```

```
    int cost = 1;

    for(int i = 0; i<threadCount;i++){
        pthread_create(&thread[i], NULL, pay, cost);
    }

    for(int i = 0; i<threadCount;i++){
        pthread_join(&thread[i], NULL, pay, cost);
    }
}
```

5. **What is a spin-lock, and why and where is it used?**
   A lock where a thread is waiting for a BUSY lock "spins" (busy/wait) in a tight loop until some other thread makes it FREE. Spinlock is used to protect shared resourses from being accessed by multiple threads or processes at the same time.

There is other synchronization methods like semaphores and mutexes, but spinlock uses a busy-wait method where a thread continuously asks for the lock until is it available. The purpose of this is too save time by avoiding overhead of blocking/pausing the thread.

The downside of spinlock is if multiple threads are spinning and threads keeps checking if the lock is available, which would waste time and CPU-cycles. To answer `where` a spinlock is used I would say that it should be used when the critical region is anticipated to be held for a short period of time. Other methods mentioned above may be more effective if the waiting period is longer or there is many threads asking for the lock.
6. **List the issues involved with thread synchronisation in multi-core architectures. Two lock algorithms are MCS and RCU (read-copy-update). Describe the problems they attempt to address. What hardware mechanism lies at the heart of each?**
Isssue involved with thread synchronisation in multi-core architectures:

- Implementations of shared objects can affect multiprocessor performance.
- Splitting a shared object into a set of related objects with their own lock can increase performance. But it can raise the issue of correctness. Even if each individual operation on a shared object is atomic, we must reason about interactions of sequences of operations across objects
- Deadlock can occur

MCS is for cases when there is a significant number of waiting threads. It uses a *spinlock* implementation where each waiting thread spins on a separate memory location. It takes advantage of a *compare-and-swap* method use to build queue of waiting threads. Lock is passed to next thread efficiently in the queue because it spins on its own queue entry.

RCU is optimized for when there are more readers than writers. Readers surrounds each critical section with calls to acquire and release the RCU lock. Since RCU is an implementation of reader/writer lock, there are three changes to the reader/writer lock:

1. Writer must *publish* its changes to the shared data structure with a single, atomic write.
2. Readers may see old or new version because RCU allows any number of concurrent reads on critical sections
3. Garbage collect after all readers complete within a *grace period*

2 Deadlocks

1. **What is the difference between resource starvation and a deadlock?**
   Starvation is the lack of progress for one task a thread is running, despite resources being available. A factor that can promote starvation is resources given to higher priority tasks.

Deadlock is a form of starvation but with a stronger condition. It is a cycle waiting amoung set of threads, where each thread waits for some other thread to take action. Thus execution is blocked. Deadlock implies starvation, but not the other way around.

2. **What are the four necessary conditions for a deadlock? Which of these are inherent properties of an operating system?**
   1 Bounded resources. There are a limited number of threads that can simultaneously use a resource.

2 No preemption. Once a thread acquires a resource, the resource cannot be revoked until the thread is done using the resource.

3 Wait while holding. A thread holds one resource while waiting for another. This condition is sometimes called multiple independent requests because it occurs when a thread first acquires one resource and then tries to acquire another.

4 Circular waiting. There is a set of waiting threads such that each thread is waiting for a resource held by another

3. **How does an operating system detect a deadlock state? What information does it have available to make this assessment?**
   There are many ways to detect a deadlock state in the system

One method is using a *graph* to illustrate the flow of threads and resources; nodes as resources and threads, and direct edges as relationship between resource and thread (wait and owned by). If only one thread can hold a resource and there is multiple resources in the graph,

there is only a deadlock if and only if there is a cycle in the graph. If there are multiple resources and multiple threads can access a resource, a cycle in the graph can lead to deadlock but is not guaranteed.

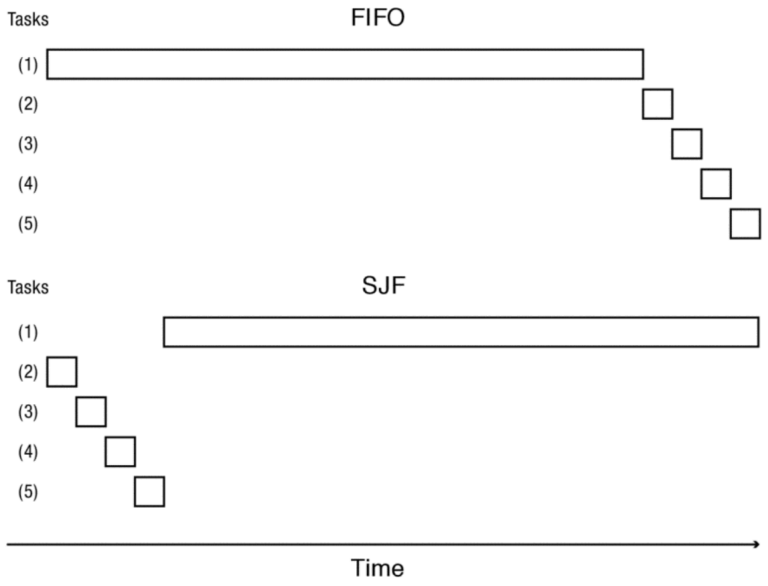
Another solution is to use a variation of *Dijkstra's Banker's algorithm by Coffman, Elphick, and Shoshani*. Here we assume to not know the max resources needed by a thread in the system, so we cannot assess if the current state is safe or if future requests can lead to deadlock. But we can use the information we have; current set of resources, granted requests, and pending requests, and ask if it is possible for the current set of requests to eventually be satisfied assuming no more requests come and all threads eventually complete. If so, there is no deadlock even if we are in an unsafe state. If not there is a deadlock.

3 Scheduling

1. **Uniprocessor scheduling**

   a. When is first-in-first-out (FIFO) scheduling optimal in terms of average response time? Why?

   FIFO is gives best average response time when there is a fixed number of tasks to complete with approximately same worksize and they only need the processor. FIFO do each task in order in which they arrive and running it until it finishes. If there is a lot of task with different sizes, the long task will take up a lot more time thus the short task will have poor response time.



   We can see here a workload of task with different sizes and how the short task have poor average response time FIFO when there is varied task size.

   b. Describe how Multilevel feedback queues (MFQ) combines first in-first-out, shortest job first, and round robin scheduling in an attempt at a fair and efficient scheduler. What (if any) are its shortcomings?

   MFQ is a scheduling algorithm with multiple priority levels managed using round *robin*

queues, where a task is moved between priority levels based on how much processing time it has used. High priority queues have shorter time quantum, while lower priority has longer time quantum. As the time quantum gets large and priority levels are low, thread have a higher chance of running to completion which approximates *FIFO*.

New tasks enter in the highest priority queue and drops down priority when time quantum expires. If a thread uses less than its quantum, it keeps its high priority or its promoted, so its finishes its job quickly. This approximates *SJF* without knowing job length.

MFQ achieves *starvation freedom* and *max-min fairness* through implementing monitoring over all process using MFQ scheduler: tasks whose processes have reached their fair share are only scheduled if all processes at the same level have also reached their fair share. Processes that receives less than its fair share will have its tasks increased in priority and vice versa.

Some *shortcomings* can be that there is a higher frequency of context switching on higher levels. A task can also attempt to keep its priority by doing a short I/O request right before its time quantum expires

2. **Multi-core scheduling**
   a. Similar to thread synchronisation, a uniprocessor scheduler running on a multi-core system can be very inefficient. Explain why (there are three main reasons). Use MFQ as an example.
   The first reason is **contention for the MFQ lock**. The lock may become a bottleneck as the number of processor increases. This depends on how much computation each thread does before blocking I/O.

The second reason is **cache coherence overhead**. Each processor will need to fetch the current state of the MFQ from the cache of the previous processor. On a uniprocessor scheduler, the scheduling data structure is likely to already be loaded into the cache. On a multiprocessor, the data structure are accessed and modified by multiple processors in turn, so the most reason version is likely to be cached by the processor that made the newest update. Fetching from remote cache takes longer than fetching from local cache. Because of this delay happens while holding the MFQ lock, the lock is held for longer period leading to more bottleneck

The last reason is **limited cache reuse:** Threads that runs on the first available processor, are more likely to be assigned to a different processor when/if they are scheduled. This implies that any data needed by the thread is less likely to be cached on that different processor. Furthermore, the most recent version of the thread's data is likely to be in a remote cache, requiring even more slowdown as the remote data is fetched into local cache
b. Explain the concept of work-stealing.
Work stealing is a scheduling technique used in a multiprocessor. Each processor has a

queue of tasks. When tasks is executed, a task may also spawn new tasks that can be executed in parallell. These tasks are initially placed on the queue which the processor takes tasks from an executes them. When a processor runs out of tasks, it looks at the queue of other processors and "*steals*" their tasks.

This method of load balancing distributes the scheduling over idle processors, and as long as processors have work to do, no scheduling overhead happens.